

Homework 2

Ravi Kini

February 18, 2026

Exercise 1

Part (a)

Let A be a symmetric 3×3 matrix:

$$A = \begin{bmatrix} a_1 & b & c \\ b & a_2 & d \\ c & d & a_3 \end{bmatrix}$$

To annihilate the c entries (in row/column 3 and column/row 1), we use a Givens rotation that acts as the identity except in rows and columns 2, 3. Letting $r = \sqrt{b^2 + c^2}$, the rotation matrix Q has the form:

$$Q = \begin{bmatrix} 1 & 0 & 0 \\ 0 & b/r & c/r \\ 0 & -c/r & b/r \end{bmatrix}$$

Then, to see that it annihilates the c entries:

$$\begin{aligned} (QAQ^T)_{1,3} &= \sum_{i=1}^3 (QA)_{1,i} Q_{i,3}^T = \sum_{i=1}^3 \sum_{j=1}^3 Q_{1,j} A_{j,i} Q_{i,3}^T \\ &= \sum_{i=1}^3 Q_{1,1} A_{1,i} Q_{i,3}^T = \sum_{i=2}^3 Q_{1,1} A_{1,i} Q_{i,3}^T \\ &= 1 \cdot b \cdot -c/r + 1 \cdot c \cdot b/r = 0 \\ (QAQ^T)_{3,1} &= \sum_{i=1}^3 (QA)_{3,i} Q_{i,1}^T = \sum_{i=1}^3 \sum_{j=1}^3 Q_{3,j} A_{j,i} Q_{i,1}^T \\ &= \sum_{i=1}^3 \sum_{j=2}^3 Q_{3,j} A_{j,i} Q_{i,1}^T = \sum_{j=2}^3 Q_{3,j} A_{j,1} Q_{1,1}^T \\ &= -c/r \cdot b \cdot 1 + b/r \cdot c \cdot 1 = 0 \end{aligned}$$

Part (b)

Let A be a symmetric $n \times n$ matrix where the leading $(n-1) \times (n-1)$ submatrix has been tridiagonalized:

$$A = \begin{bmatrix} a_1 & b_2 & & & & & \\ b_2 & a_2 & b_3 & & & & \\ & b_3 & a_3 & \ddots & & & \\ & & \ddots & \ddots & b_{n-1} & d & \\ & & & b_{n-1} & a_{n-1} & c & \\ & & & d & c & a_n & \end{bmatrix}$$

To annihilate the d entries (in row/column n and column/row $n-2$), we use a Givens rotation that acts as the identity except in rows and columns $n-1, n$.

Letting $r = \sqrt{b_{n-1}^2 + d^2}$, the rotation matrix Q has the form:

$$Q = \begin{bmatrix} I_{n-2} & 0 & 0 \\ 0 & b_{n-1}/r & d/r \\ 0 & -d/r & b_{n-1}/r \end{bmatrix}$$

We see it is orthogonal, and that the product $Q A Q^T$ would have the same leading $(n-2) \times (n-2)$ submatrix. Then, to see that it annihilates the d entries:

$$\begin{aligned} (Q A Q^T)_{n-2,n} &= \sum_{i=1}^n (Q A)_{n-2,i} Q_{i,n}^T = \sum_{i=1}^n \sum_{j=1}^n Q_{n-2,j} A_{j,i} Q_{i,n}^T \\ &= \sum_{i=1}^n Q_{n-2,n-2} A_{n-2,i} Q_{i,n}^T = \sum_{i=n-1}^n Q_{n-2,n-2} A_{n-2,i} Q_{i,n}^T \\ &= 1 \cdot b_{n-1} \cdot -d/r + 1 \cdot d \cdot b_{n-1}/r = 0 \\ (Q A Q^T)_{n,n-2} &= \sum_{i=1}^n (Q A)_{n,i} Q_{i,n-2}^T = \sum_{i=1}^n \sum_{j=1}^n Q_{n,j} A_{j,i} Q_{i,n-2}^T \\ &= \sum_{i=1}^n \sum_{j=n-1}^n Q_{n,j} A_{j,i} Q_{i,n-2}^T = \sum_{j=n-1}^n Q_{n,j} A_{j,n-2} Q_{n-2,n-2}^T \\ &= -d/r \cdot b_{n-1} \cdot 1 + b_{n-1}/r \cdot d \cdot 1 = 0 \end{aligned}$$

Exercise 2

Let T be a symmetric tridiagonal $n \times n$ matrix, with QR factorization $T = QR$. Consider the matrix RQ . We see that:

$$\begin{aligned} R &= Q^T T \\ RQ &= Q^T T Q \\ (RQ)^T &= (Q^T T Q)^T \\ &= Q^T T^T Q = Q^T T Q = RQ \end{aligned}$$

Since $(RQ)^T = RQ$, we see that RQ is symmetric.

Now note that in QR factorization, column k of Q is column k of A orthogonalized with respect to the previous columns, and so can only have nonzero components in the span of those columns (and itself). Since the first k columns of A have zeros in rows i where $i > k+1$, it follows that column k of Q must also have a zero in such rows. Equivalently, $Q_{i,j} = 0$ for $i > j+1$. In other words, Q is zero below the first subdiagonal. Further, since R is upper triangular, it is zero below the diagonal ($R_{i,j} = 0$ for $i > j$). We now consider $(RQ)_{i,j}$, with $i > j+1$:

$$\begin{aligned} (RQ)_{i,j} &= \sum_{k=1}^n R_{i,k} Q_{k,j} \\ &= \sum_{k=i}^n R_{i,k} Q_{k,j} \\ &= \sum_{k=i}^n R_{i,k} \cdot 0 = 0 \end{aligned}$$

Since $(RQ)_{i,j} = 0$ for $i > j+1$, it follows that RQ is zero below the first subdiagonal. Since RQ is symmetric, it must also be zero above the first superdiagonal, which means that RQ is tridiagonal symmetric.

Exercise 3

Consider the shifted QR iteration algorithm. Let $\underline{Q}_k = Q_1 Q_2 \dots Q_k$ and $\underline{R}_k = R_k R_{k-1} \dots R_1$. We proceed by induction. For the base case where $k = 1$, we see that:

$$\begin{aligned} Q_1 R_1 &= A - \mu_1 I \\ A_1 &= R_1 Q_1 + \mu_1 I \\ &= Q_1^T Q_1 R_1 Q_1 + \mu_1 I \\ &= Q_1^T (A - \mu_1 I) Q_1 + \mu_1 I \\ &= Q_1^T A Q_1 - \mu_1 I + \mu_1 I = Q_1 A Q_1 \end{aligned}$$

Now assume that for some k , $A_k = \underline{Q}_k^T A \underline{Q}_k$ and $(A - \mu_k I) \dots (A - \mu_1 I) = \underline{Q}_k \underline{R}_k$. Then:

$$\begin{aligned} Q_{k+1} R_{k+1} &= A_k - \mu_{k+1} I \\ \underline{Q}_{k+1} \underline{R}_{k+1} &= \underline{Q}_k Q_{k+1} R_{k+1} \underline{R}_k \\ &= \underline{Q}_k (A_k - \mu_{k+1} I) \underline{R}_k \\ &= \underline{Q}_k (\underline{Q}_k^T A \underline{Q}_k - \mu_{k+1} I) \underline{R}_k = (A \underline{Q}_k - \mu_{k+1} \underline{Q}_k I) \underline{R}_k \\ &= (A - \mu_{k+1} I) \underline{Q}_k \underline{R}_k \\ &= (A - \mu_{k+1} I) (A - \mu_k I) \dots (A - \mu_1 I) \end{aligned}$$

Further:

$$\begin{aligned} A_{k+1} &= R_{k+1} Q_{k+1} + \mu_{k+1} I \\ &= Q_{k+1}^T Q_{k+1} R_{k+1} Q_{k+1} + \mu_{k+1} I \\ &= Q_{k+1}^T (A_k - \mu_{k+1} I) Q_{k+1} + \mu_{k+1} I \\ &= Q_{k+1}^T A_k Q_{k+1} - \mu_{k+1} I + \mu_{k+1} I \\ &= Q_{k+1}^T \underline{Q}_k^T A \underline{Q}_k Q_{k+1} \\ &= \underline{Q}_{k+1}^T A \underline{Q}_{k+1} \end{aligned}$$

By induction, it follows that $A_k = \underline{Q}_k^T A \underline{Q}_k$ and $(A - \mu_k I) \dots (A - \mu_1 I) = \underline{Q}_k \underline{R}_k$ for all k .

Exercise 4

Part (a)

```
1 import math
2 EPS = 1e-15
3
4 def eye(n):
5     return [[float(i == j) for i in range(n)] for j in range(n)]
6
7 def copy(A):
8     return [row[:] for row in A]
9
10 def givens_left(A, i, j, c, s):
11     n = len(A)
12     for k in range(n):
13         a, b = A[i][k], A[j][k]
14         A[i][k] = c * a + s * b
15         A[j][k] = -s * a + c * b
16
17 def givens_right(A, i, j, c, s):
18     n = len(A)
19     for k in range(n):
20         a, b = A[k][i], A[k][j]
21         A[k][i] = c * a - s * b
22         A[k][j] = s * a + c * b
23
24 def tridiagonalize(A):
25     n = len(A)
26     T = copy(A)
27     for i in range(n - 2):
28         for j in range(n - 1, i + 1, -1):
29             b = T[i + 1][i]
30             c = T[j][i]
31             if abs(c) < EPS:
32                 continue
33             r = math.hypot(b, c)
34             givens_left(T, i + 1, j, b / r, c / r)
35             givens_right(T, i + 1, j, b / r, -c / r)
36     return T
```

Part (b)

```
1 def QR(T):
2     n = len(T)
3     Q = eye(n)
4     R = copy(T)
5     for k in range(n - 1):
6         a = R[k][k]
7         b = R[k + 1][k]
8         if abs(b) < EPS:
9             continue
10        r = math.hypot(a, b)
11        givens_left(R, k, k + 1, a / r, b / r)
12        givens_right(Q, k, k + 1, a / r, -b / r)
13    return Q, R
```

Part (c)

```
1 def leading(A):
2     n = len(A)
3     return [row[:-1] for row in A[:-1]]
4
5 def zeros(n):
6     return [[0.0 for i in range(n)] for j in range(n)]
7
8 def wilkinson(T):
9     n = len(T)
10    a, b, c = T[n - 2][n - 2], T[n - 2][n - 1], T[n - 1][n - 1]
11    delta = (a - c) / 2
12    sign = 1.0 if delta >= 0 else -1.0
13    return c - sign * b ** 2 / (abs(delta) + math.hypot(delta, b))
14
15 def QR_iteration(T, tol=1e-10, max_iter=1000):
16     n = len(T)
17     T = copy(T)
18     eigs = []
19     while n > 1:
20         for _ in range(max_iter):
21             if abs(T[n - 1][n - 2] / (abs(T[n - 2][n - 2]) + abs(T[n - 1][n - 1]))) < tol:
22                 eigs.append(T[n - 1][n - 1])
23                 T = leading(T)
24                 n -= 1
25                 break
26             mu = wilkinson(T)
27             for i in range(n):
28                 T[i][i] -= mu
29             Q, R = QR(T)
30             T = zeros(n)
31             for i in range(n):
32                 for j in range(max(0, i - 1), min(n, i + 2)):
33                     for k in range(i, min(n, j + 2)):
34                         T[i][j] += R[i][k] * Q[k][j]
35             for i in range(n):
36                 T[i][i] += mu
37     eigs.append(T[0][0])
38     return sorted(eigs, reverse=True)
```